



FitPay

INTERNAL SECURITY REVIEW

— MOOLRE STARTUP CUP 2026 — PRE-LAUNCH REVIEW

Security Posture Report

A full audit of FitPay's authentication, authorization, payment, and data-handling surfaces — and the hardening completed as a result.

- All identified critical & high-severity findings resolved and deployed

PREPARED FOR
FitPay — Accra, Ghana

REPORT DATE
8 July 2026

REVIEW TYPE
Full-surface code audit

01

Executive Summary

Ahead of launch, FitPay's full application surface was reviewed for security vulnerabilities: every API route, the authentication and authorization layer, database row-level security policies, the payment and refund money paths, secrets handling, and platform configuration.

The review found **seven critical and high-severity issues** — including a privilege-escalation path that let any client grant themselves trainer-level access, an unauthenticated relay that could send SMS under FitPay's own sender ID, and a gap that allowed a client to trigger a real mobile-money refund with no trainer approval. Every finding has since been fixed, deployed, and independently verified against the live production environment.

Alongside the fixes, several defense-in-depth measures were added: security response headers, PII redaction in server logs, fail-closed authentication on all internal service-to-service calls, and hardened secret handling using Supabase Vault. As of this report, FitPay's security posture reflects standard practice for a fintech-adjacent consumer application handling payments and health data.

7

FINDINGS FIXED

5

CRITICAL SEVERITY

~25

API ROUTES REVIEWED

10

DB TABLES + STORAGE AUDITED

Scope of review

The audit covered: every route under `/api` , including payments, disbursements, refunds, sessions, AI triggers, and webhooks; `middleware.ts` route protection; Supabase row-level security policies across all ten application tables and the avatar storage bucket; the client signup and role-assignment flow; the Moolre payment webhook and its idempotency guarantees; the repository and git history for accidentally committed secrets; and platform-level configuration including HTTP response headers.

02

Findings & Fixes

Every finding below has been fixed in code, deployed to production, and — where applicable — verified with a live test against the running application.

Critical severity

F-01 Privilege escalation: any client could self-promote to trainer

CRITICAL

FIXED

RISK Database policy did not restrict which role a user could assign to their own account, or block changing it later. Any logged-in client could grant themselves trainer access — exposing every client's personal and medical data, plus the ability to process refunds and withdraw platform funds.

FIX Self-service signup can now only create client accounts; a database trigger makes a user's role permanently immutable after account creation. Trainer accounts are provisioned separately, outside the public signup flow.

F-02 Open SMS relay under FitPay's sender ID

CRITICAL

FIXED

RISK The internal SMS-sending endpoint had no authentication. Anyone who found the URL could send arbitrary text messages through FitPay's approved Moolre sender ID — a cost and brand-abuse risk (phishing, spam).

FIX The endpoint now requires a private, server-only shared secret on every call and refuses all requests without it (fail-closed, not fail-open).

F-03 Unauthenticated AI & progression endpoints

CRITICAL

FIXED

RISK Four internal endpoints that trigger AI-generated content and workout progression had no authentication at all. Any caller could burn AI budget on demand, mass-message every active client, or silently overwrite a specific client's workout plan.

FIX Server-triggered routes now require the same internal shared secret as the SMS endpoint; the browser-triggered onboarding route now requires a valid client session and only ever acts on that client's own account.

F-04 Refund approval could be bypassed entirely

CRITICAL

FIXED

RISK A legacy endpoint let a client trigger a full, real mobile-money refund to themselves directly — no trainer review, and no check on how many sessions had already been used.

FIX The endpoint has been removed entirely. Refunds now flow only through the request → trainer-review → approval workflow, with the refund amount pro-rated to unused sessions.

F-05 Cross-tenant withdrawal of platform funds**CRITICAL****FIXED**

RISK Introduced with multi-trainer support: the available-balance calculation summed revenue across every trainer's clients, while only subtracting the requesting trainer's own withdrawals — allowing one trainer to withdraw funds earned by another trainer's clients.

FIX The balance calculation is now scoped strictly to the requesting trainer's own client revenue.

High severity**F-06 Payment webhook could be forged if misconfigured****HIGH****FIXED**

RISK If the webhook signing secret was ever left unset in production, the payment webhook would skip signature verification entirely — letting a forged "payment confirmed" callback activate a session package for free.

FIX The webhook now fails closed: it refuses to process any callback unless the configured secret is present and matches exactly.

F-07 Booking calendar leaked availability across trainers**HIGH****FIXED**

RISK Also introduced with multi-trainer support: a client's booking calendar checked availability against every trainer's bookings globally, so one trainer's schedule could incorrectly block time slots on a completely different trainer's calendar.

FIX Availability is now scoped to the client's own assigned trainer, matching the same scoping already used by the booking-conflict check.

03

Defense-in-Depth Hardening

Beyond fixing the findings above, the following measures were added so a single mistake is less likely to become an incident.

✓ **Security response headers**

X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, and HTTP Strict Transport Security are now set on every response, closing off clickjacking and MIME-sniffing classes of attack.

✓ **Fail-closed internal authentication**

Every server-to-server call between FitPay's own routes now requires a private shared secret. If the secret is ever missing from the environment, those routes refuse to run rather than silently allowing the call.

✓ **PII redaction in logs**

Payment transfer logs previously recorded full provider responses, including client names and phone numbers. Logging was narrowed to status codes only.

✓ **Idempotent payment webhook**

Verified the existing design: every webhook call is matched against a unique payment reference before any state changes, so Moolre's automatic retries can never double-activate a purchase.

✓ **Row-level security verified**

Confirmed RLS is enabled with an explicit policy on every one of the ten application tables, plus the avatar storage bucket, so the database itself enforces access control independent of the application layer.

✓ **Secret hygiene**

Scanned the repository and its full git history for accidentally committed credentials — none were found. Broadened .gitignore so no future .env file can be committed by accident.

Secrets & infrastructure

Two categories of secret are now required for the application to operate, both fail-closed by design — meaning a missing secret disables the feature rather than silently exposing it:

SECRET	PURPOSE	STORED IN
INTERNAL_API_SECRET	Authenticates FitPay's own server-to-server calls (SMS sending, AI triggers, progression engine, and the scheduled monthly client check-in)	Vercel environment variables; Supabase Vault for the scheduled job
MOOLRE_WEBHOOK_SECRET	Verifies that payment confirmation callbacks genuinely originate from Moolre	Vercel environment variables
SUPABASE_SERVICE_ROLE_KEY	Server-only database access that bypasses row-level security for trusted backend operations	Vercel environment variables (never exposed client-side)

04

Verification & Deployment Status

Fixes were not just written — they were deployed and tested against the live application. The scheduled monthly client check-in was manually triggered end-to-end: the request was authenticated with the production secret, reached the correct route, generated a message via the AI trigger, and delivered a real SMS through Moolre — confirming the entire authenticated call chain works correctly in production.

Deployment checklist

ITEM	STATUS
Database migration — multi-trainer data model	Applied
Database migration — security hardening (role lock, immutability)	Applied
Database migration — scheduled monthly check-in job	Applied
Moolre webhook signing secret configured	Set
Internal API secret configured (application + scheduler)	Verified live
Production domain confirmed and consistent across config	Verified
End-to-end scheduled job test (auth → AI → SMS delivery)	Passed

05

Honest Limitations & Next Steps

This was a thorough internal code review, not a substitute for an independent third-party penetration test. The following are known gaps, intentionally left open, that are reasonable to defer for an early-stage launch but worth tracking as FitPay grows.

No rate limiting on public endpoints

Login, signup, and payment initiation currently have no request-throttling. Low risk at pilot scale with a small, known user base; worth adding before wider public growth to blunt brute-force and abuse attempts.

No automated security regression testing

Fixes were verified manually and by direct code review. There is no CI check today that would catch a future change accidentally re-opening one of these findings (for example, a new endpoint shipped without the internal-secret guard).

No independent third-party audit performed

This review was conducted internally. For a platform handling payments and health data at meaningful scale, a professional external penetration test is recommended as a future milestone, not a launch blocker.

Taken together, the findings identified in this review represented meaningful risk to client data, trainer funds, and platform trust — and all have been resolved and confirmed working in production as of the date of this report. FitPay's current security posture reflects sound practice for its stage: authenticated by default, fail-closed on missing configuration, and enforced at both the application and database layers.